Table of contents

- Chapter I: introduction to OAuth2 with Keycloak
  - The nuts and bolts of OAuth2
    - Authorization code flow
    - Service account flow
    - Access token usage
    - Refresh token usage
  - Keycloak
    - Starting up
    - Realms
    - Clients
      - Creating a confidential client in Keycloak

Chapter I: introduction to OAuth2 with Keycloak

We mentioned that OAuth2 is a protocol, a *standard*, not something we can run: OAuth2 is a specification, not an application.
To leverage the OAuth2 standard we must use a OAuth2-compliant **identity management provider** (IDP), which is fancy for "a system composed by one or more servers that all run an application that does the authorization and authentication instead of us".
One of the leading IDPs is Keycloak, developed and maintained by RedHat, which also happens to be completely open source and self-hostable. Other than that, Keycloak has a vibrant and large community, definitely a plus when it comes to open source self-hostable projects.

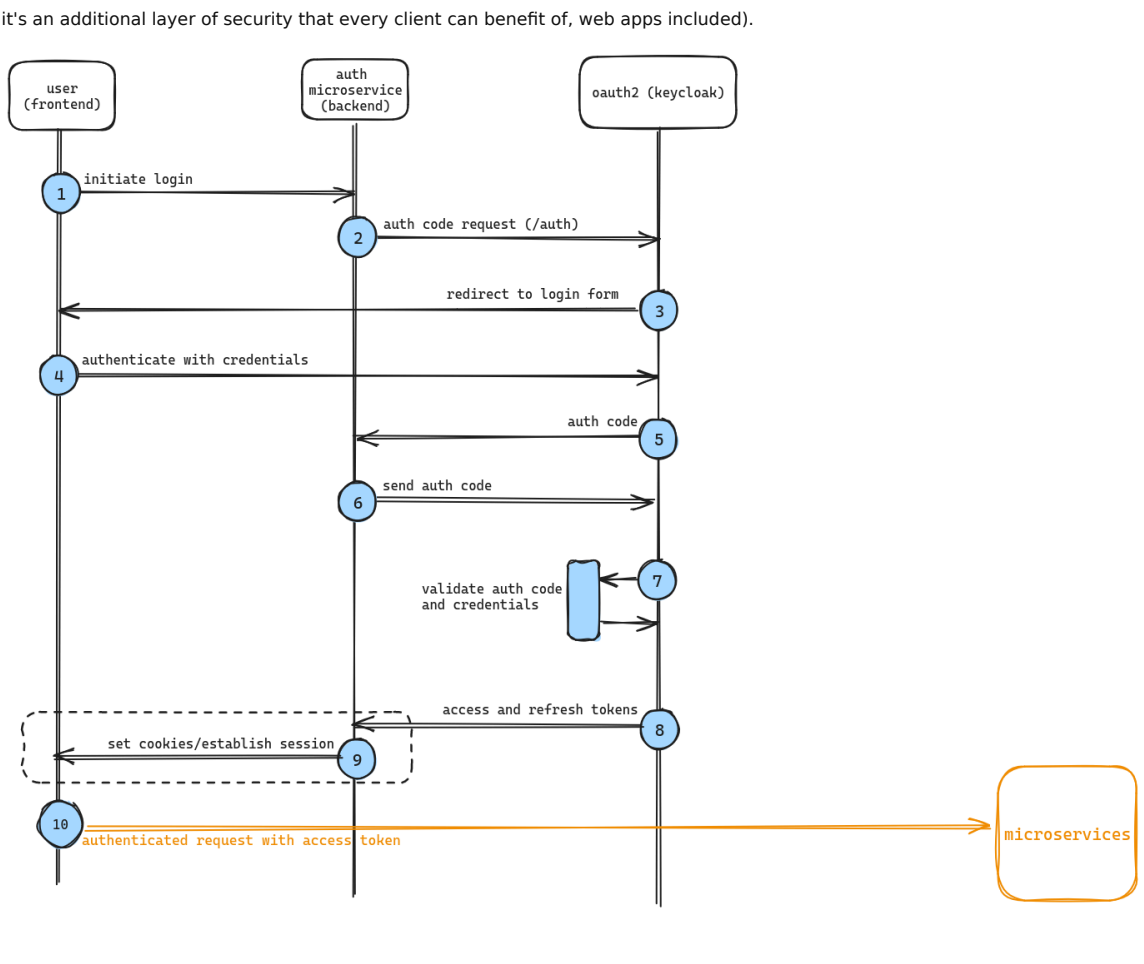
The nuts and bolts of OAuth2

How do we effectively gain access to resources protected by an OAuth2 authorization scheme? The answer is simple: through a **flow**, a standard set of steps that the IDP will go through while authenticating a client's identity.
The main flows that we will use in our journey through this project are the **authorization code flow** and the **service account flow**.
The first is used when the application that is requesting access to a protected resource is a **web** or **mobile** application, or really any application that is **user-driven**, whereas the latter is used when we need to authenticate **requests made from other programs that are not user-driven** (i.e. a microservice!).
Both of these flows will output two, very important, pieces of information: the **`access token`** and the **`refresh token`**, which are the foundations to OAuth2's authorization system.

- The `access token` is used to **issue authenticated requests to our microservices**. It's our passport **into the system** and it has a very short lifespan: this is a key characteristic of access tokens; tokens that grant **access** to protected resources *should* have a short TTL, because **tokens leak**, security protocols are **breached**, things always take the wrong turn with microservices. For these reasons, access tokens should live as short as possible, because, if stolen, they will grant access to the system on behalf of the user that got the token in the first place. This token can be `Opaque`, a random, meaningless string, or a `JWT` that encodes information. We will use a `JWT`.
- The `refresh token` is used to **get new access tokens when they expire**. They are (very) long-lived with respect to **access tokens** and, needless to say, they are equally important and must be stored carefully and safely.

Authorization code flow

Here's a picture that describe how the user gets into the system with the authorization code flow.
We will use the "basic" version, without PKCE, because of simplicity and also because PKCE is mainly related to mobile apps (but it's an additional layer of security that every client can benefit of, web apps included).

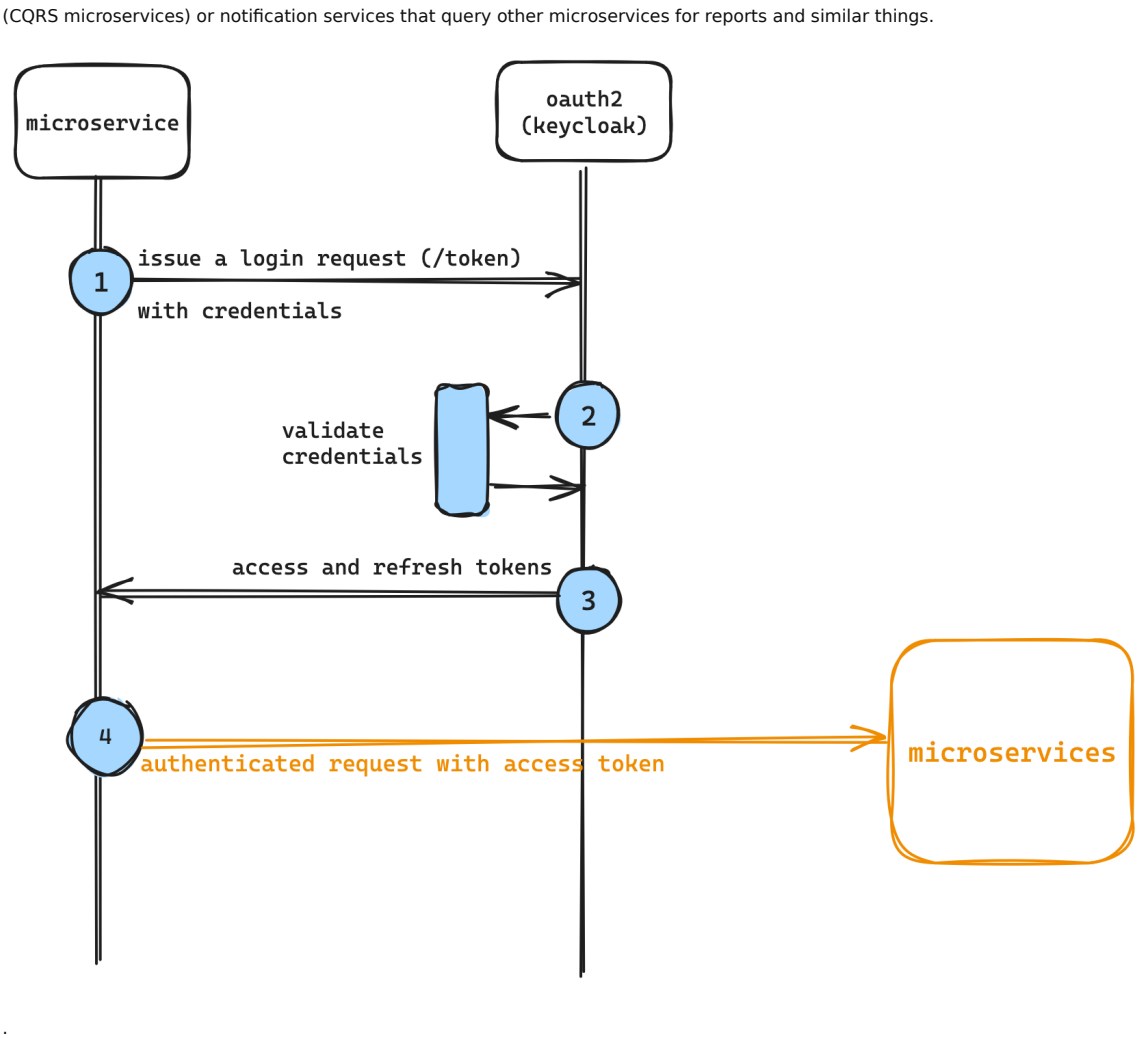


user (frontend)
auth microservice (backend)
oauth2 (keycloak)
1 initiate login
2 auth code request (/auth)
3 redirect to login form
4 authenticate with credentials
5 auth code
6 send auth code
7 validate auth code and credentials
8 access and refresh tokens
9 set cookies/establish session
10 authenticated request with access token
microservices

1. the user clicks on the login button (or follows a login link);
2. the backend issues an authorization code request, this starts a redirection process to the login form. The authorization code will make sure that the request for the access and refresh token is not forged.
3. the user gets redirected to the login form;
4. the user authenticates;
5. if the credentials are correct, the auth code is issued and sent to the backend;
6. once the auth code is received, the backend issues another request to retrieve the tokens. Note, again, that the auth code's presence is critical to mitigate security risks: by issuing an auth code, the client **never directly contacts the OAuth2 server** and the authentication flow is completely delegated to the server. This enforces security and doesn't allow privileged access to the OAuth2 server from potentially vulnerable clients; imagine what would happen if the authorization code didn't pass through the server, but was instead passed to a vulnerable client: if the authorization code gets intercepted by an attacker, the entire authorization session would be stolen, and malicious third parties could authenticate on the vulnerable client's behalf. Instead, it's the backend that requests credentials with the authorization code and the OAuth2 server is configured to accept authentication requests from the server only, thus, hardening security;
7. the OAuth2 server validates the code and the credentials;
8. the tokens are issued and...
9. ...a session is issued between the backend and the frontend;
10. the frontend can now make authenticated requests to other microservices;

Service account flow

As already mentioned, the service account flow is used by non-user driven processes, i.e. applications that aggregate user data (CQRS microservices) or notification services that query other microservices for reports and similar things.



microservice
oauth2 (keycloak)
1 issue a login request (/token) with credentials
2 validate credentials
3 access and refresh tokens
4 authenticated request with access token
microservices

.

1. the microservice has some pre-generated credentials with which it will make a request for tokens;
2. the OAuth2 service validates the credentials...
3. ...and returns the tokens to the microservice;
4. authenticated requests can now be sent to other microservices.

Notice how the flow is much shorter and simpler: that's because one can make stronger assumptions about the microservice, which means that it can be presumed much less likely to be vulnerable with respect to malicious third parties.
Nevertheless, credentials on the microservice must be safely handled and rotated periodically.

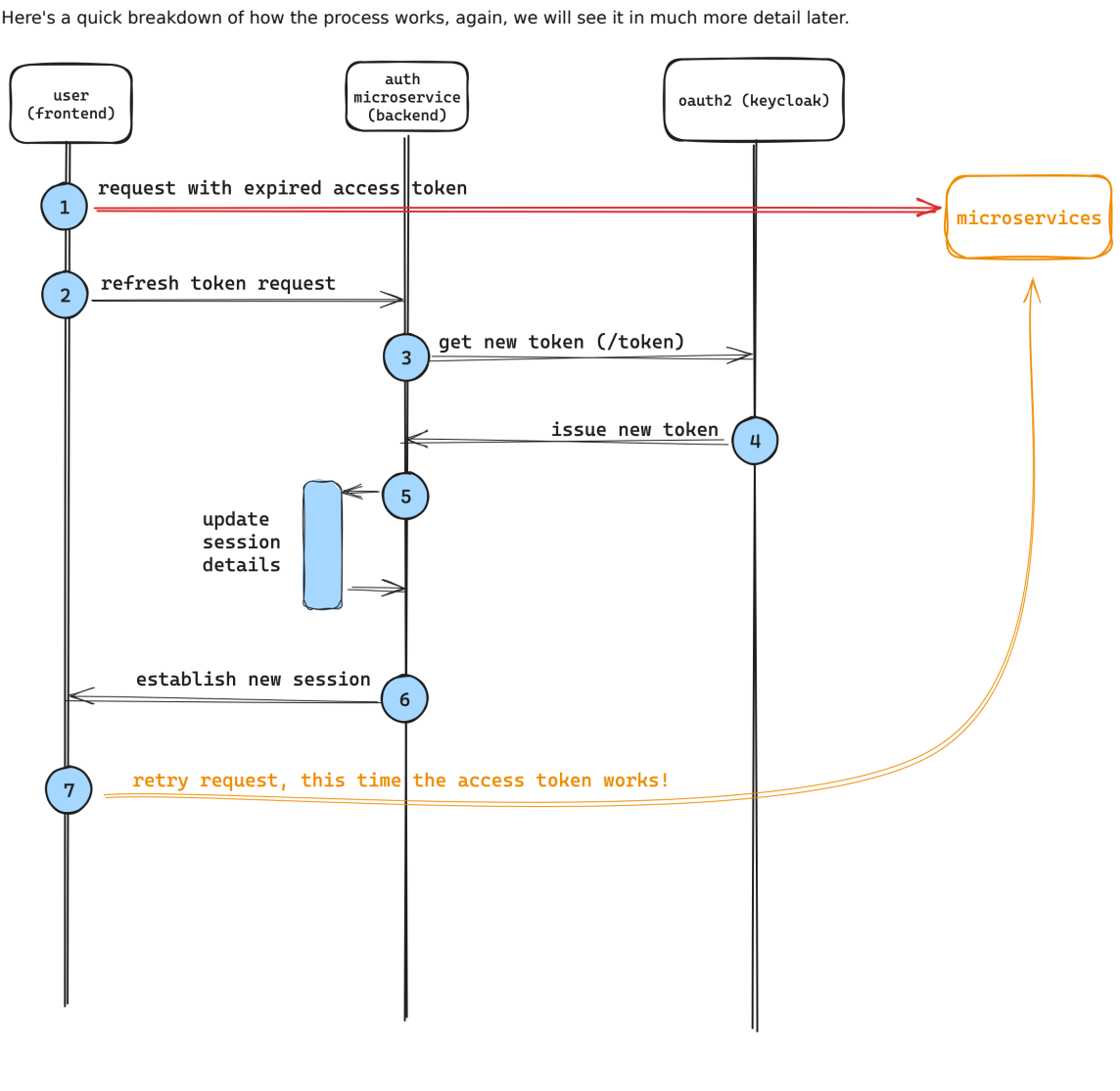
Access token usage

Once the access token is obtained, it can be used to make authenticated request towards the protected API (in the picture above we can represent the protected API with the "microservices" block).
It is the microservice's responsibility, then, to verify the token's validity.
For simplicity's sake, we are not going to delve into cryptographical details on how JWTs are created: we just need to know that JWTs are cryptographically signed strings that encode information in JSON format. Don't worry too much about them, we will delve deeper into this topic later in Chapter IV.
The signing happens on the OAuth2 server's side with a private key and each microservice that will need to validate tokens issued by this server will receive a copy of the server's public key: this way, the verification is done completely "offline", meaning that microservices won't need to interrogate the OAuth2 server every time it receives an authenticated request.

We will see in detail how this process works in the next chapters, for now that's all we need to know.

Refresh token usage

Once the access token expires, verifying the JWT will return an error. That's when the client should **refresh** its access token.
The refresh mechanism is very simple: the OAuth2 server exposes a refresh token endpoint to which applications can request new access tokens.
Note that here, too, the process happens entirely on the backend side (for the same reasons we talked about before) and it's the auth microservice's (the one represented in the first image of this section) responsibility to ensure the token is refreshed correctly.
Here's a quick breakdown of how the process works, again, we will see it in much more detail later.



user (frontend)
auth microservice (backend)
oauth2 (keycloak)
1 request with expired access token
microservices
2 refresh token request
3 get new token (/token)
4 issue new token
5 update session details
6 establish new session
7 retry request, this time the access token works!

Keycloak

Now that we talked about the theory, let's put it into practice with Keycloak.
We will use Keycloak's Docker image because we will model our system through a Docker Compose file for simplicity and reproducibility.

Starting up

Let's fire up a basic Keycloak instance: to do so, use the following `compose.yml` file

```
keycloak:
    build:
      context: .
      dockerfile: ./path/to/Dockerfile
      network: host
    entrypoint: ["/opt/keycloak/bin/kc.sh", "start-dev"]
    environment:
      - KC_HOSTNAME=localhost
      - KC_HOSTNAME_PORT=8080
      - KC_HOSTNAME_STRICT=false
      - KC_HOSTNAME_STRICT_HTTPS=false
      - KEYCLOAK_ADMIN=admin
      - KEYCLOAK_ADMIN_PASSWORD=admin
    ports:
      - 8080:8080
```

Keycloak requires a Dockerfile to start, because it needs to generate a private/public keypair for crypto operations:

```
FROM quay.io/keycloak/keycloak:latest as builder

# Enable health and metrics support
ENV KC_HEALTH_ENABLED=true
ENV KC_METRICS_ENABLED=true

WORKDIR /opt/keycloak
# for demonstration purposes only, please make sure to use proper certificates in production instead
RUN keytool -genkeypair -storepass password -storetype PKCS12 -keyalg RSA -keysize 2048 -dname "CN=server" -alias server -ext "SAN:c=DNS:localhost,IP:127.0.0.1" -keystore conf/server.keystore
RUN /opt/keycloak/bin/kc.sh build

FROM quay.io/keycloak/keycloak:latest
COPY --from=builder /opt/keycloak/ /opt/keycloak/
```

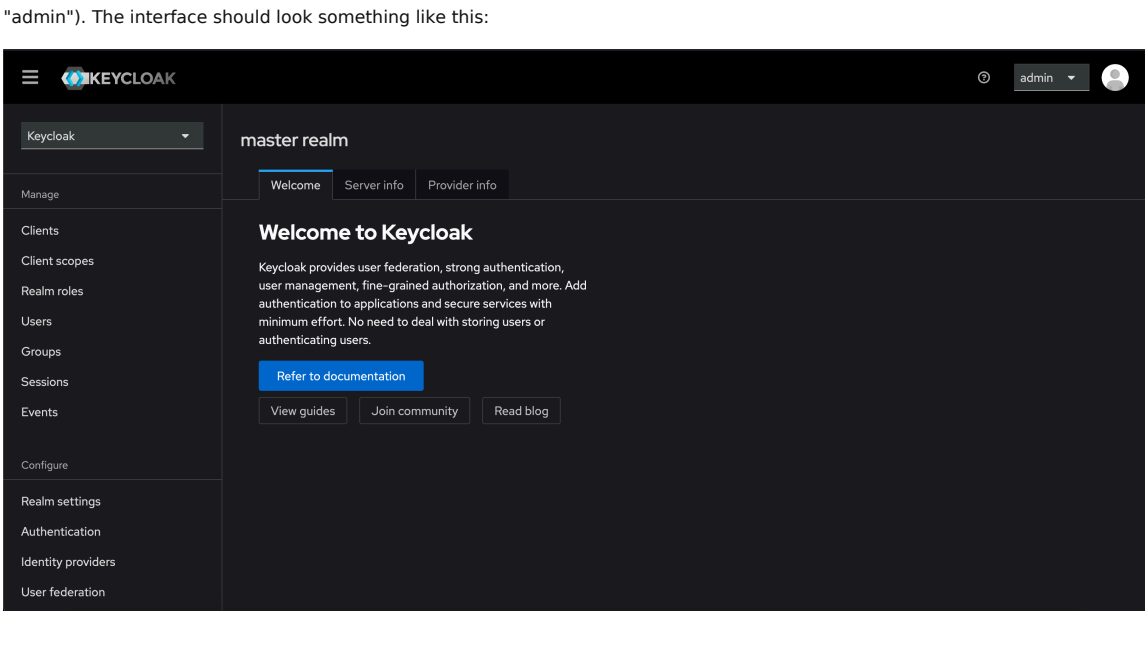
Everything is ready, we just need to start our container:

```
docker compose up
```

Note: we are not going to delve into the details of a production-ready deployment with Keycloak because it's out of the scope of this project. This setup will be the same we are going use throughout the whole project and, of course, it's not production ready since it uses an in-memory database to store user information, which is not ideal for a highly available environment.

Realms

Now that we fired up our Keycloak instance, let's navigate to http://localhost:8080 and login with the default credentials ("admin", "admin"). The interface should look something like this:



KEYCLOAK
admin
Keycloak
Manage
Clients
Client scopes
Realm roles
Users
Groups
Sessions
Events
Configure
Realm settings
Authentication
Identity providers
User federation
Welcome
Server info
Provider info
Refer to documentation
View guides
Join community
Read blog

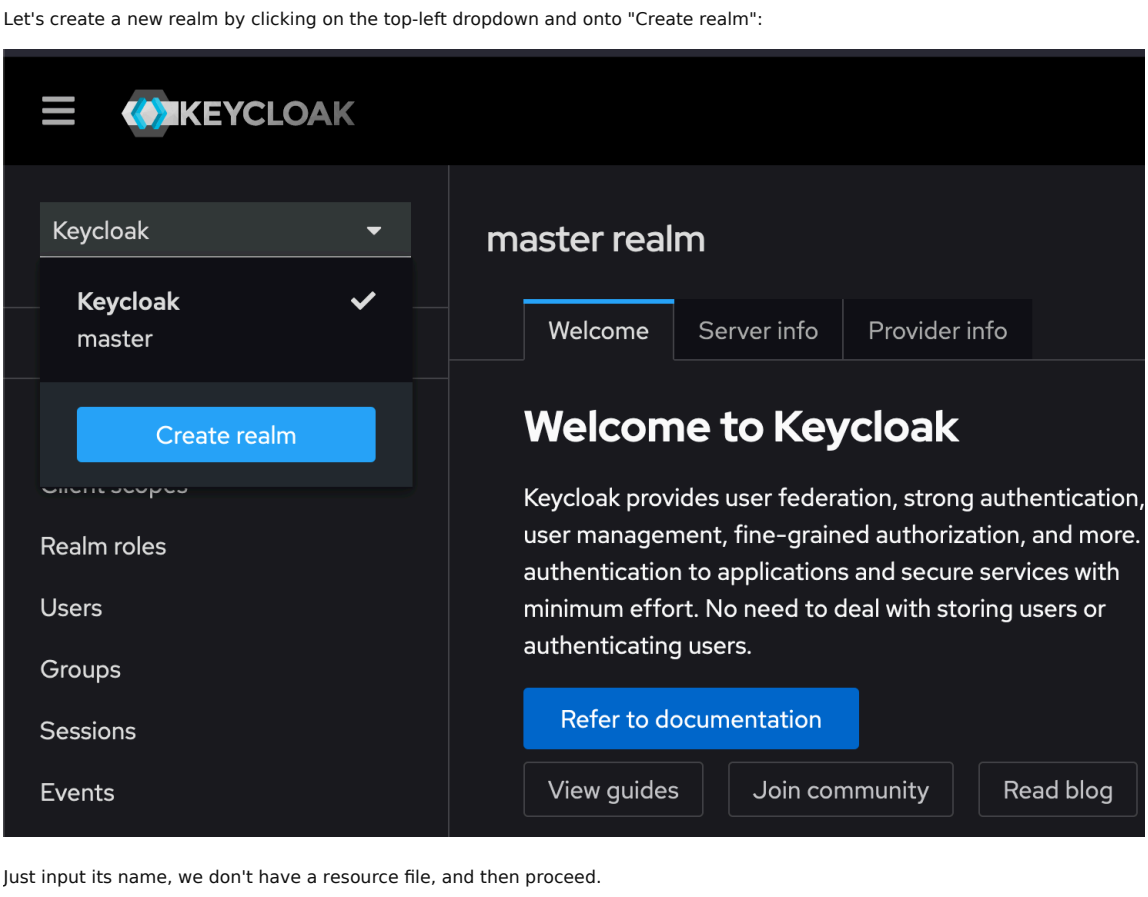
master realm

Welcome to Keycloak

Keycloak provides user federation, strong authentication, user management, fine-grained authorization, and more. Add authentication to applications and secure services with minimum effort. No need to deal with storing users or authenticating users.

Keycloak introduces the concept of **realms** as a way to *namespace* different configurations and users.
A realm in Keycloak is essentially a **tenant**, which is a space where all the information about a set of users and applications is managed. Each realm is isolated from other realms, allowing for multi-tenancy within a single Keycloak instance.
Let's imagine we were to have an application that runs two environments: staging and production. We certainly wouldn't want to have staging and production users mixed, so we would have to create two realms (i.e. *staging-realm* and *production-realm*) to separate the users.
The staging environment would then refer to the first realm, whereas the production one would use the latter.

Keycloak starts, by default, with the **master** realm already created, which is a management realm, used to create and administrate other realms. You should not use this realm for production usage, because access to this realm can compromise the instance's configuration.
Let's create a new realm by clicking on the top-left dropdown and onto "Create realm":

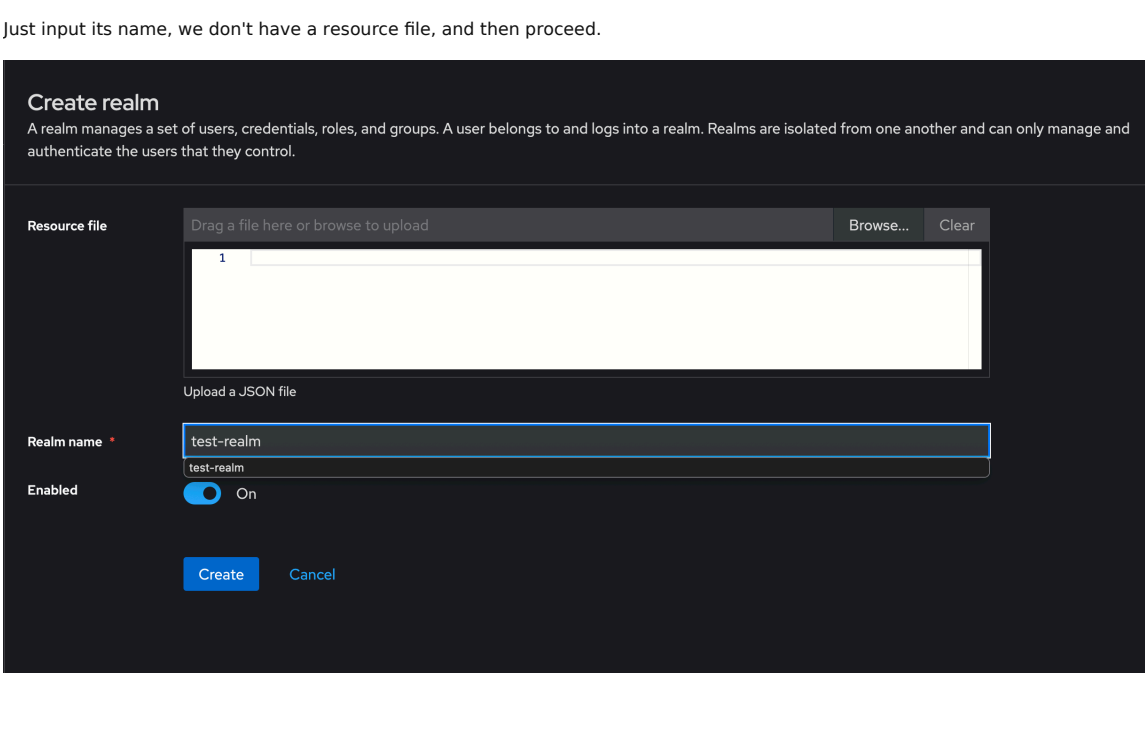


KEYCLOAK
Keycloak
Keycloak
master
Create realm
Realm roles
Users
Groups
Sessions
Events
master realm
Welcome
Server info
Provider info
Refer to documentation
View guides
Join community
Read blog

Welcome to Keycloak

Keycloak provides user federation, strong authentication, user management, fine-grained authorization, and more. authentication to applications and secure services with minimum effort. No need to deal with storing users or authenticating users.

Just input its name, we don't have a resource file, and then proceed.



Drag a file here or browse to upload
Browse...
Clear
Upload a JSON file
Create
Cancel

Create realm

A realm manages a set of users, credentials, roles, and groups. A user belongs to and logs into a realm. Realms are isolated from one another and can only manage and authenticate the users that they control.

Resource file

Realm name * test-realm

Enabled ● On

Clients

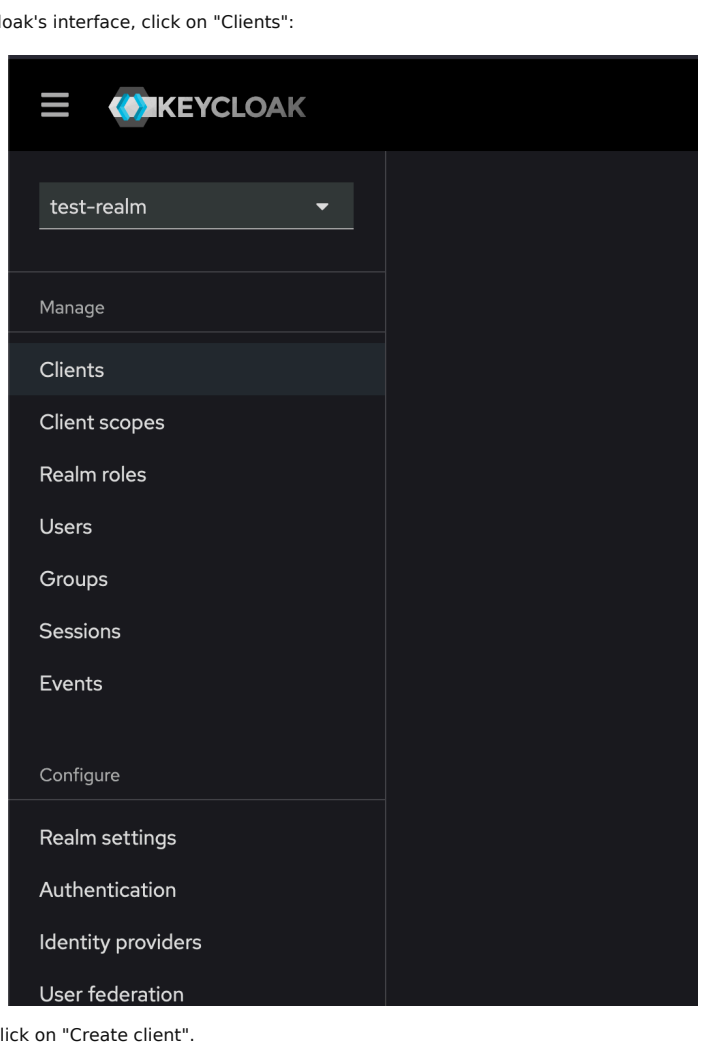
Let's talk about how applications can interact with Keycloak, meaning, how can applications perform all the flows and tasks we mentioned before?
Keycloak, and, in general, all OAuth2 servers, introduces the concept of *clients*, that represents something, an application, trying to use OAuth2 functionality to access a protected resources.
There are **confidential clients** and **public** clients:

- confidential clients are applications that are able to securely authenticate with the authorization server. In our previous example, the confidential client is the auth microservice. Basically, confidential clients are the middlemen between the user and the authorization server. Confidential clients are represented by a `client_secret` and a `client_id`;
- public clienst are basically applications running in a browser or on a mobile device. They are essentialy the end users of access and refresh tokens.

In other words, public clients rely on confidential clients to obtain credentials and then they use the latter to access private resources.
Let's see how to create a client and how to use them.

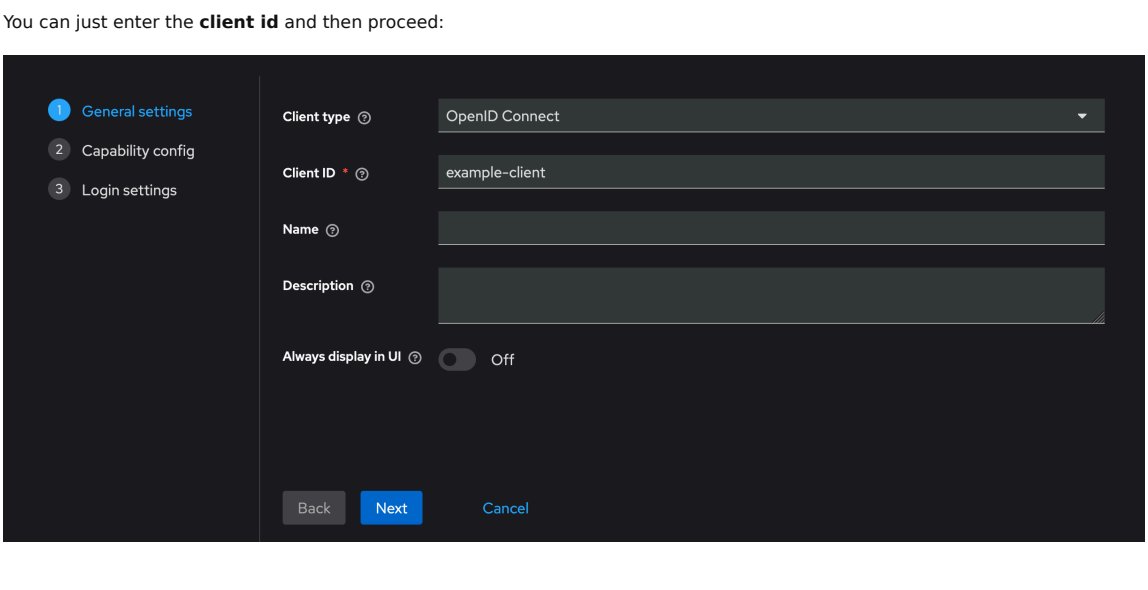
Creating a confidential client in Keycloak

On the left panel of Keycloak's interface, click on "Clients":



KEYCLOAK
test-realm
Manage
Clients
Client scopes
Realm roles
Users
Groups
Sessions
Events
Configure
Realm settings
Authentication
Identity providers
User federation

In the following screen, click on "Create client".
You can just enter the **client id** and then proceed:



1 General settings
2 Capability config
3 Login settings
Back
Next
Cancel

Client type: OpenID Connect

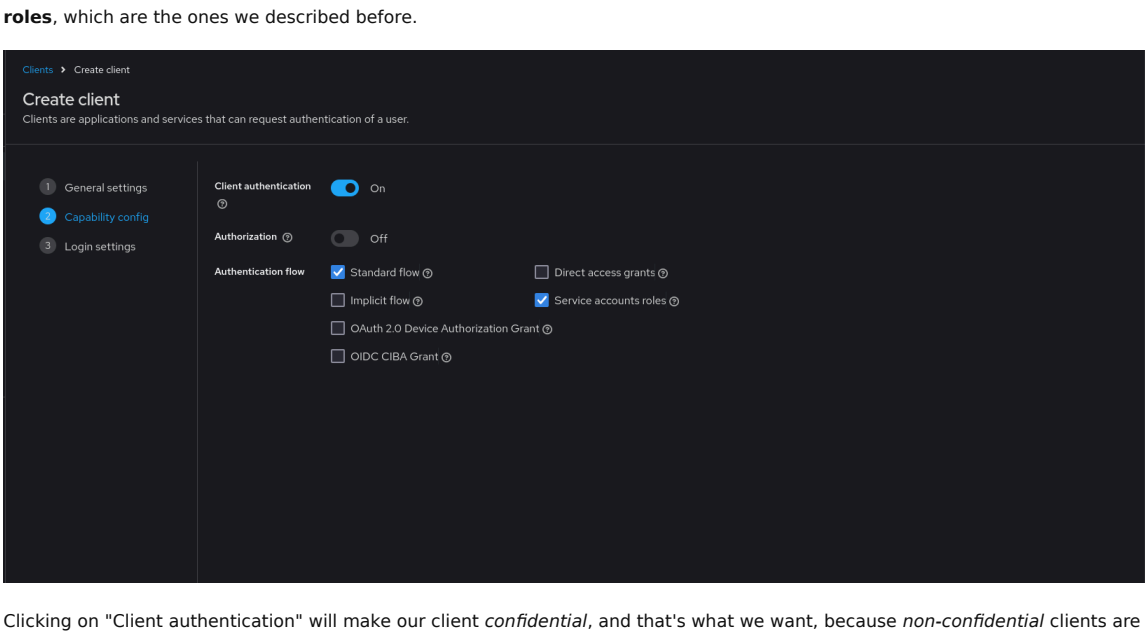
Client ID *: example-client

Name

Description

Always display in UI: Off

It's now time to select which flows our client will support. The most common are the **Standard flow** and the **Service accounts roles**, which are the ones we described before.



Clients > Create client
1 General settings
2 Capability config
3 Login settings

Create client

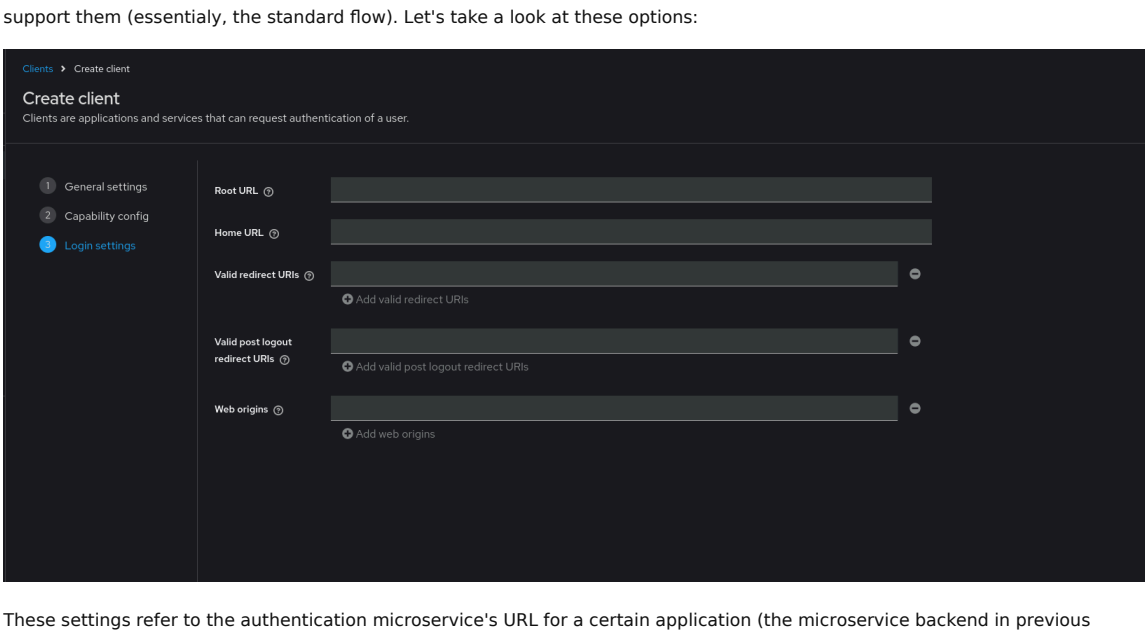
Clients are applications and services that can request authentication of a user.

Client authentication: On

Authorization: Off

Authentication flow:
- ☑ Standard flow
- ☐ Direct access grants
- ☐ Implicit flow
- ☑ Service accounts roles
- ☐ OAuth 2.0 Device Authorization Grant
- ☐ OIDC CIBA Grant

Clicking on "Client authentication" will make our client *confidential*, and that's what we want, because *non-confidential* clients are deprecated (still supported for legacy purposes).
The "Login settings" screen is very important, because here we can define the callback URLs for our authorization flows that support them (essentialy, the standard flow). Let's take a look at these options:



Clients > Create client
1 General settings
2 Capability config
3 Login settings
Add valid redirect URIs
Add valid post logout redirect URIs
Add web origins

Create client

Clients are applications and services that can request authentication of a user.

Root URL

Home URL

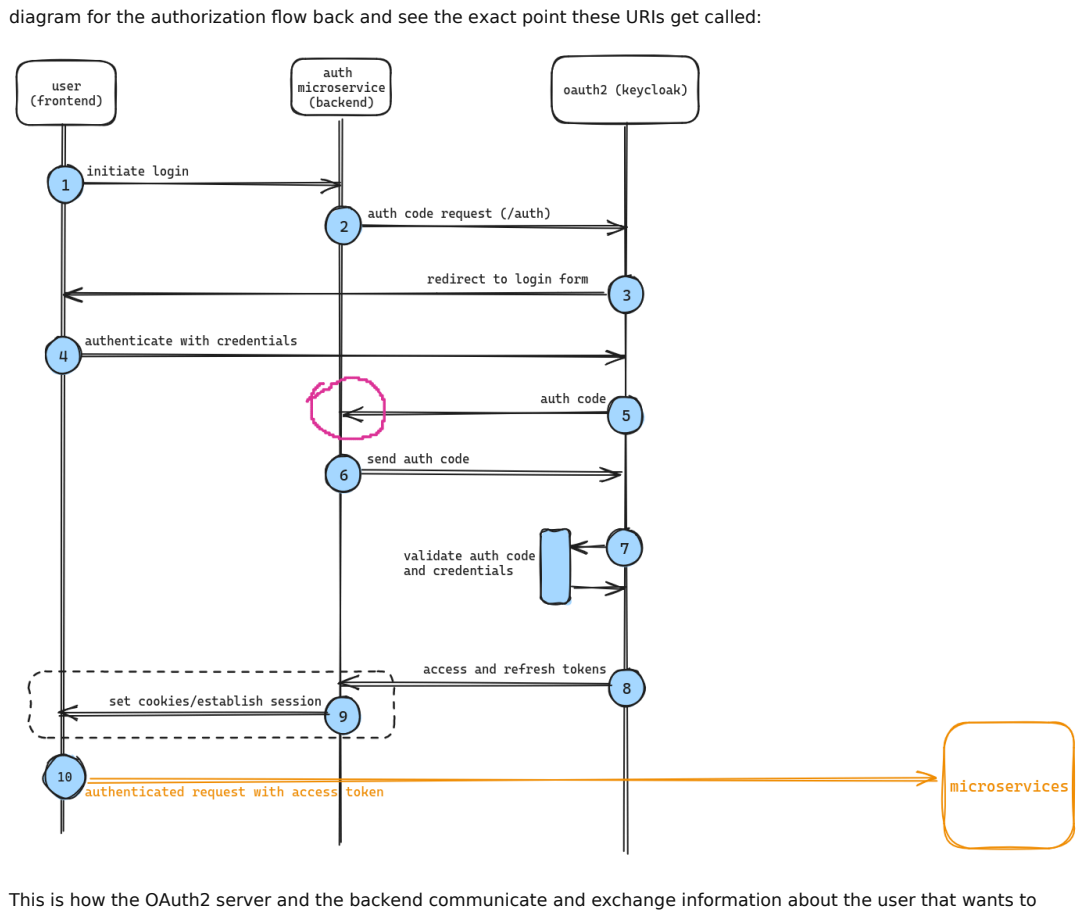
Valid redirect URIs

Valid post logout redirect URIs

Web origins

These settings refer to the authentication microservice's URL for a certain application (the microservice backend in previous picture):

- Root URL refers to the backend's main entry point, basically the domain name without any path attached to it;
- Home URL refers to the application's homepage, if any;
- Valid redirect URIs refer to the URIs we want to use when authenticating a user with our OAuth2 server. Let's bring the diagram for the authorization flow back and see the exact point these URIs get called:



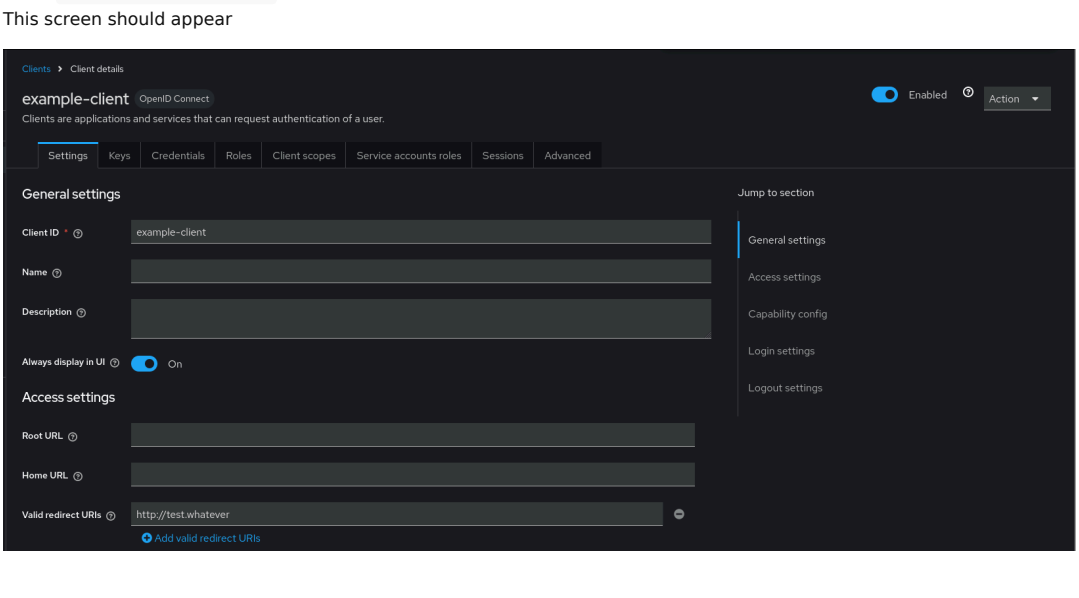
user (frontend)
auth microservice (backend)
oauth2 (keycloak)
1 initiate login
2 auth code request (/auth)
3 redirect to login form
4 authenticate with credentials
5 auth code
6 send auth code
7 validate auth code and credentials
8 access and refresh tokens
9 set cookies/establish session
10 authenticated request with access token
microservices

  This is how the OAuth2 server and the backend communicate and exchange information about the user that wants to login. Redirect URIs are called, basically, when the OAuth2 server is done authenticating a user and is ready to grant tokens to a certain user. Generally, endpoints specified between these are called with a GET request and a `code` query parameter is passed. This code is the "auth code" we see in the picture and it is exchanged between the backend and the OAuth2 server for the reasons we mentioned before (keeping the client out of the loop for safety reasons). There is the possibility to enter multiple URIs because it might be necessary to support different login flows, but it's generally normal to have just one;
- Valid post logout redirect URIs are endpoints that are called after the user log outs and usually represent the "goodbye" screen you see after logging out of a website;
- Web origins represent the CORS origins allowed to make requests to our OAuth2 server.

In this screen, each field is optional, but at least one valid redirect URI is required if we want to use the standard flow. For now, we don't have an application to redirect codes to, so we can enter whatever we want.

Even if we don't have an application, we can see how the *standard flow* works by simulating it:

1. Enter `http://test.whatever` or whatever you want inside the "Valid redirect URIs" field and proceed creating the client. This screen should appear

   Clients > Client details
   OpenID Connect
   Enabled
   Action
   Settings
   Keys
   Credentials
   Roles
   Client scopes
   Service accounts roles
   Sessions
   Advanced
   Jump to section
   General settings
   Access settings
   Capability config
   Login settings
   Logout settings
   Add valid redirect URIs

   example-client

   Clients are applications and services that can request authentication of a user.

   General settings

   Client ID: example-client

   Name

   Description

   Always display in UI: On

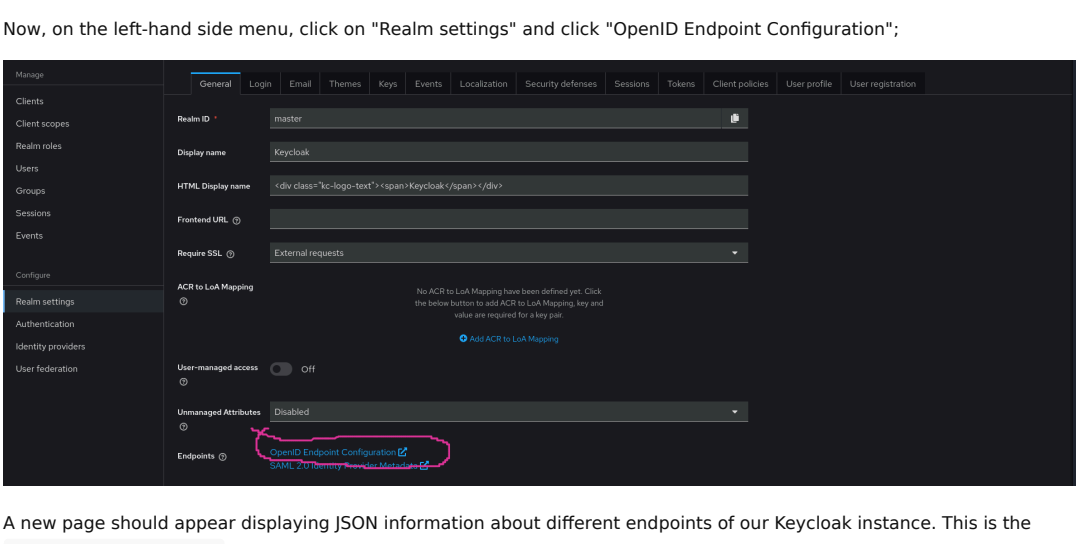
   Access settings

   Root URL

   Home URL

   Valid redirect URIs: http://test.whatever

2. Let's click on "Credentials" and copy the "Client Secret" field. This is what we will use to make requests to Keycloak when authenticating someone or an application;
3. Now, on the left-hand side menu, click on "Realm settings" and click "OpenID Endpoint Configuration";

   Manage
   Clients
   Client scopes
   Realm roles
   Users
   Groups
   Sessions
   Events
   Configure
   Realm settings
   Authentication
   Identity providers
   User federation
   General
   Login
   Email
   Themes
   Keys
   Events
   Localization
   Security defenses
   Sessions
   Tokens
   Client policies
   User profile
   User registration
   Add ACR to LoA Mapping
   OpenID Endpoint Configuration
   SAML 2.0 Identity Provider Metadata

   Realm ID *: master

   Display name: Keycloak

   HTML Display name: `<div class="kc-logo-text"><span>Keycloak</span></div>`

   Frontend URL

   Require SSL: External requests

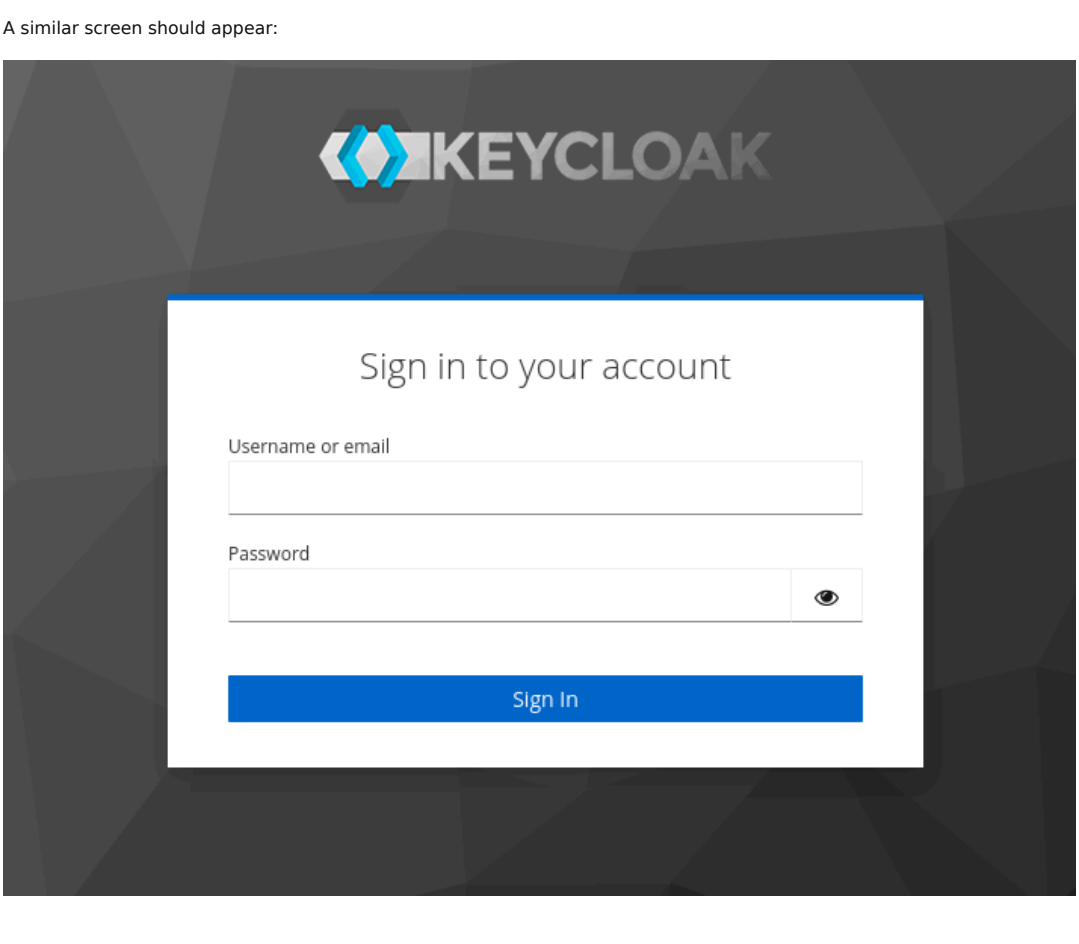
   ACR to LoA Mapping

   User-managed access: Off

   Unmanaged Attributes: Disabled

   Endpoints

   A new page should appear displaying JSON information about different endpoints of our Keycloak instance. This is the `openid-configuration` endpoint, that displays various URLs you can request to do various things concering our Keycloak instance. A notable one is the `authorization_endpoint`, which is the URL that gets called when a client wants to login to our system. Another interesting one is the `token_endpoint`, which is the endpoint the auth backend will call in point 6 of the previous diagram;
4. Let's copy the `authorization_endpoint` and start a fictional login flow;
5. To start a login flow, we need to call the `authorization_endpoint` with to **mandatory** query parameters: `response_type=code` and `client_id=example-client`. The first parameter means that we want to obtain an auth code out of this login flow, and the latter indicates for which confidential client we want this login flow to start for. An example request would be: `http://localhost:8080/realms/test-realm/protocol/openid-connect/auth?response_type=code&client_id=example-spring`;
6. Open an incognito window and paste the URL above. We open an incognito window because we are already logged into Keycloak with our admin account and navigating to the URL above would skip the next step;
7. A similar screen should appear:

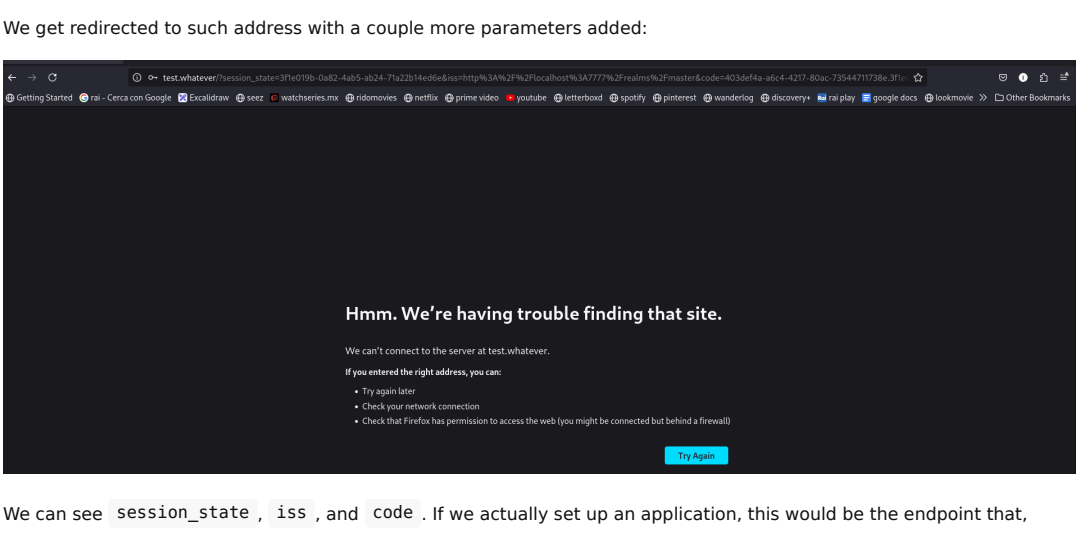
   KEYCLOAK
   Sign in

   Sign in to your account

   Username or email

   Password

8. We can login with our admin account for the moment, we will see how to create users in a bit;
9. After logging in we should be redirected to the redirect URI we specified before, which, in our case, should be `http://test.whatever`;
10. We get redirected to such address with a couple more parameters added:

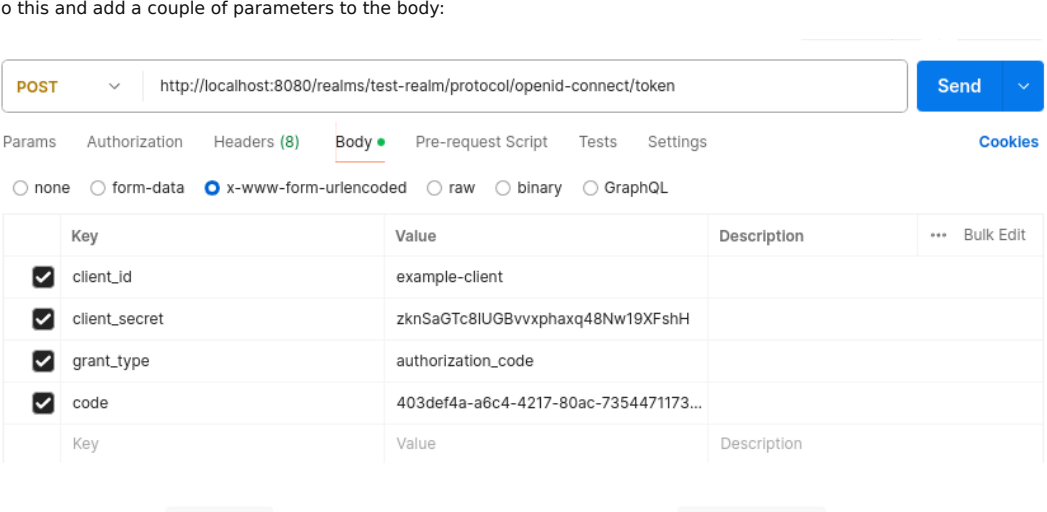
    

    Hmm. We're having trouble finding that site.

    We can't connect to the server at test.whatever.

    If you entered the right address, you can:
    - Try again later
    - Check your network connection
    - Check that Firefox has permission to access the web (you might be connected but behind a firewall)

    We can see `session_state`, `iss`, and `code`. If we actually set up an application, this would be the endpoint that, when called, would make the request to obtain the access and refresh tokens (step 6 of the diagram);
11. We can simulate this request with Postman, an HTTP client;
12. To obtain a token we need to make a `POST` request to the `token_endpoint` we talked about in step 3 of this list. Let's do this and add a couple of parameters to the body:

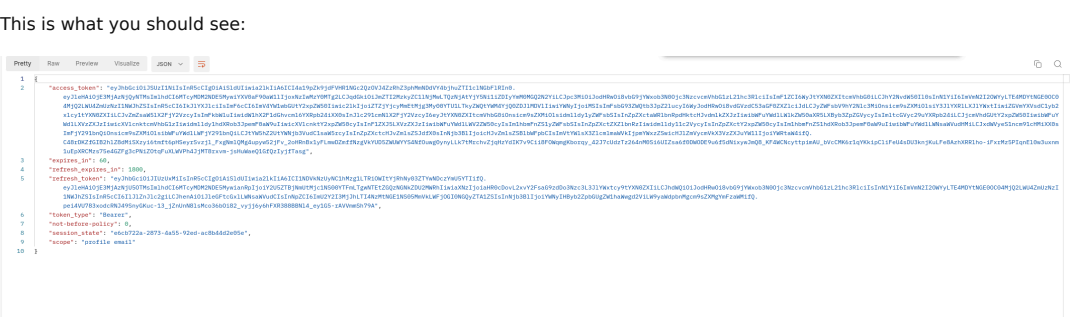
    Send
    Params
    Authorization
    Headers (8)
    Body
    Pre-request Script
    Tests
    Settings
    Cookies
    Bulk Edit

    POST http://localhost:8080/realms/test-realm/protocol/openid-connect/token

    ○ none ○ form-data ● x-www-form-urlencoded ○ raw ○ binary ○ GraphQL

	Key	Value	Description
☑	client_id	example-client	
☑	client_secret	zknSaGTc8lUGBvvxphaxq48Nw19XFshH	
☑	grant_type	authorization_code	
☑	code	403def4a-a6c4-4217-80ac-7354471173...	
	Key	Value	Description

    - we specify the `client_id` we are making the request for, along with its `client_secret`;
    - we specify the `grant_type`, which is the type of credentials we want to get. In this case it's `authorization_code`;
    - let's also add the `code` we received when we got redirected to `http://test.whatever` in step 10 of this list;
    - let's click on "Send" and wait for the response.
13. This is what you should see:

14. We got to the end of the standard flow! Nice! We have everything we need to make authenticated requests to our microservices, if we had any.

Note: we will use Postman throughout the entirety of chapters III - V. We will then add a frontend and see how the redirect URIs come into place.
I know this might all be confusing for first-time readers, but don't worry: I was too when I first learned about all this stuff! Don't sweat it, everything will fall into place by the end of this guide!

Next chapter: Chapter II: the system and its components — how everything connects